

Reporte Ejecutivo

Proyecto Final - Machine Learning con PySpark y Docker - 2026-I

Estudiante: Sergio Huertas

Repositorio: github.com/chechitoooo/proyecto_final_Huertas_Sergio

1. Resumen ejecutivo

Este proyecto trabaja con dos fuentes de datos colombianas. La primera es el dataset TUSO del catastro de Bogotá, que tiene 1.060.001 predios con tres columnas: código del lote, tipo de uso y área en metros cuadrados. La segunda es un corpus de 204 comentarios ciudadanos sobre alcaldías, ministerios, TransMilenio y servicios públicos. Los hallazgos principales son tres: el 68,1 % de los predios de Bogotá son de uso residencial con un área promedio de 187 metros cuadrados; el área del predio por sí sola explica más del 95 % de la predicción en los modelos de clasificación, con un AUC de 0,79; y en el análisis de sentimiento, el modelo preentrenado de HuggingFace entiende sarcasmo y ambigüedad que el enfoque clásico de TF-IDF no logra capturar.

2. Qué hice en cada bloque y por qué

Bloque 1. Análisis exploratorio sobre TUSO. Cargué el dataset en PySpark (1.060.001 filas) y lo exploré con seis tareas. Clasifiqué los predios por tamaño (pequeño, mediano, grande), calculé estadísticas descriptivas, revisé si había nulos o duplicados (no había) e identifiqué valores atípicos. También hice dos gráficos con Matplotlib: un histograma de áreas y un gráfico de barras por tipo de uso. Usé PySpark en vez de pandas porque un millón de registros no caben cómodos en memoria local.

Bloque 2. Machine learning sobre TUSO. Construí un pipeline con StringIndexer, OneHotEncoder, VectorAssembler y StandardScaler. Elegí StandardScaler en lugar de MinMaxScaler porque los predios tienen áreas enormes (hasta 2,8 millones de m²) que aplastarían la escala del MinMax, y porque PCA necesita datos centrados en cero. En la parte no supervisada apliqué PCA y K-Means con el método del codo (9 clústeres). En la parte supervisada definí una variable objetivo binaria: predio residencial (si USOTUSO es 1) o no residencial (el resto). Entrené Regresión Logística y Random Forest, reporté métricas (accuracy, precision, recall, F1 y AUC), hice matriz de confusión y extraje la importancia de variables del Random Forest. Por último corrí validación cruzada con CrossValidator sobre 9 combinaciones de hiperparámetros.

Bloque 3. NLP sobre corpus institucional. Armé un corpus de 204 documentos simulando tweets reales sobre instituciones públicas colombianas (TransMilenio, alcaldías, ministerios, policía, salud). Tokenicé con RegexTokenizer y filtré 179 stop words en español. Reporté estadísticas del corpus: total de tokens, vocabulario, TTR y cuántas palabras aparecen una sola vez (hapax legomena). Vectoricé con CountVectorizer más IDF y comparé los rankings de frecuencia cruda contra TF-IDF. Luego usé el modelo preentrenado de HuggingFace (robertuito, de la Universidad de Buenos Aires) para etiquetar el corpus como positivo o negativo. Sobre esas etiquetas entrené Regresión Logística con vectores TF-IDF. Por último comparé los dos enfoques con 8 casos difíciles de sarcasmo, frases ambiguas y negaciones.

3. Resultados clave

Hallazgo 1 (Bloque 1). De los 1.060.001 predios, 721.963 (el 68,1 %) son de uso residencial, con un área promedio de 187,13 m². La media general del dataset es 295,56 m² pero la mediana es apenas 158,7 m², lo que muestra que unos pocos predios enormes jalan la media hacia arriba. El máximo registrado es 2.799.947,8 m² y hay 6.046 predios con más de 5.000 m². Estos datos justifican usar logaritmo del área en el Bloque 2 para que los outliers no distorsionen los modelos.

Hallazgo 2 (Bloque 2). El área manda en la predicción. En Random Forest, USOAREA aporta el 39,6 % de importancia y log_area el 40,1 %. Juntas suman el 79,7 %. El sector geográfico apenas aporta un 4,3 %. Ambos modelos (Regresión Logística y Random Forest) dan resultados parecidos: accuracy 0,78, F1 0,85 y AUC 0,78. Donde más se equivoca el modelo es en los predios no residenciales: el 60,1 % de los que no son residenciales los clasifica mal como si lo fueran. La validación cruzada dio como ganador Random Forest con 100 árboles y profundidad máxima de 15 (AUC en CV de 0,7878).

Hallazgo 3 (Bloque 3). El modelo TF-IDF con Regresión Logística sacó accuracy de 0,60 y F1 de 0,71, pero está muy sesgado hacia la clase negativa: acertó los 15 negativos del test pero erró los 10 positivos. Las palabras que más pesan para positivo son "gracias", "histórico" y "excelente". Las que más pesan para negativo son "caos", "desastre", "corrupción" y "prometió". En los 8 casos difíciles, HuggingFace acertó en frases con sarcasmo como "Qué maravilla, otro día sin agua" (la detectó como negativa) mientras que TF-IDF no pudo con ese tipo de lenguaje. El corpus tiene un 71 % de hapax legomena, es decir que casi tres de cada cuatro palabras distintas aparecen una sola vez, algo muy típico de tweets cortos.

4. Cómo se conectan los tres bloques

Los tres bloques cuentan una historia. El Bloque 1 me obligó a conocer los datos antes de modelar: vi que el área tiene una cola muy larga hacia la derecha y que el 68 % de los predios son residenciales. Esa observación me llevó a aplicar logaritmo en el Bloque 2, y funcionó: con solo cuatro variables el Random Forest llegó a un AUC de 0,79. Además el Bloque 2 me mostró que el área lo explica casi todo (80 % de importancia) y que el sector geográfico no aporta casi nada (4 %). Eso es valioso para alguien que trabaje en catastro: no necesita datos complejos para clasificar predios, con el área basta. El Bloque 3 me llevó al otro extremo: los textos. Ahí el área no sirve de nada y lo que importa es el contexto. Un modelo clásico como TF-IDF falla con sarcasmo y frases ambiguas; un transformer como robertuito las entiende. La conclusión práctica es que no hay un único modelo que sirva para todo. Si voy a hacer avalúos catastrales, necesito un modelo interpretable como Random Forest, donde pueda explicar por qué clasifiqué cada predio. Si voy a monitorear redes sociales para una alcaldía, necesito un transformer que entienda el tono real de los ciudadanos, aunque pese 500 MB y no pueda explicar cada decisión. Un sistema maduro usaría los dos.

5. Limitaciones honestas de mi análisis

Muestras pequeñas. En el Bloque 2 trabajé con 100.000 registros en vez del millón completo porque mi computador tiene 8 GB de RAM y apenas 1,6 GB libres. Los 9 clústeres y las métricas reflejan esa muestra, no necesariamente todo Bogotá. En el Bloque 3 el corpus es sintético: lo armé yo simulando tweets reales, pero no fue validado por otras personas ni refleja toda la diversidad de opiniones que hay en redes sociales colombianas.

Etiquetas prestadas. Las etiquetas de sentimiento del Bloque 3 las puso robertuito, no un humano. Si el modelo preentrenado tiene sesgos (por ejemplo, clasificar como neutro algo que un colombiano leería como negativo), ese sesgo se transmite al clasificador TF-IDF. Para estar seguros habría que anotar manualmente al menos 500 ejemplos y comparar.

Pocas variables en TUSO. El dataset solo trae tres columnas. Si tuviera datos como el avalúo catastral, el estrato, el año de construcción o coordenadas geográficas (todas disponibles en el IDECA), los modelos del Bloque 2 probablemente superarían un AUC de 0,85 sin mucho esfuerzo.

6. Qué le diría a alguien que quiera seguir este trabajo

Use el Dockerfile que dejé en el repositorio. Con docker-compose up se levanta Jupyter con Spark 3.5.0 y todas las dependencias listas. Ejecute los notebooks en orden: primero el de EDA, luego el de ML y por último el de NLP.

Para mejorar los resultados: reemplace el corpus sintético del Bloque 3 por tweets reales, mínimo 2.000. Cruce TUSO con otras fuentes del IDECA para meter más variables al modelo. Pruebe XGBoost de Spark ML, que en datos tabulares suele ganarle a Random Forest. Y si tiene acceso a un clúster con varios núcleos, corra PCA y K-Means sobre el dataset completo, no sobre la muestra. Para producción, una buena estrategia es usar HuggingFace para etiquetar datos nuevos automáticamente y luego reentrenar un modelo TF-IDF ligero que corra rápido y barato.